

GestiónApp

Documentación de la aplicación y explicación de funcionamiento

Proyecto: examen-des-app-web

Fecha: 14/6/2026

1. Descripción general

GestiónApp es una aplicación de una sola página (SPA) desarrollada con Vue.js 3 y Vue Router. Utiliza Bootstrap 5 para el diseño y Axios para conectarse a una API falsa de MockAPI. La aplicación permite autenticación básica y la gestión de usuarios y productos.

2. Estructura principal del proyecto

Los archivos y carpetas más importantes son los siguientes:

- src/main.js: punto de entrada de la aplicación y registro del router.
- src/App.vue: componente raíz que contiene la barra de navegación y el router-view.
- src/router/index.js: definición de rutas y guards de autenticación.
- src/services/api.js: servicios Axios para usuarios y productos.
- src/views/LoginView.vue: vista de inicio de sesión.
- src/views/UsuariosView.vue: CRUD de usuarios usando modales.
- src/views/ProductosView.vue: CRUD de productos con tarjetas y modales.
- src/components/AlertaComponent.vue: componente de alertas reutilizable.
- src/assets/global.css: estilos globales de la aplicación.

3. Rutas y navegación

La aplicación tiene tres rutas principales: login, usuarios y productos. El acceso a usuarios y productos está protegido y requiere un token en sessionStorage.

- /login: vista de autenticación.
- /usuarios: vista de gestión de usuarios, protegida por auth.
- /productos: vista de gestión de productos, protegida por auth.

```
const routes = [  
  { path: '/', redirect: '/login' },  
  { path: '/login', name: 'Login', component: LoginView },  
  { path: '/usuarios', name: 'Usuarios', component: UsuariosView, meta:  
    { requiresAuth: true } },  
  { path: '/productos', name: 'Productos', component: ProductosView, meta:  
    { requiresAuth: true } }  
];
```

4. Funcionalidad de autenticación

El login solicita todos los usuarios desde MockAPI y valida el username y password. Si la credencial es correcta, se guarda un token falso en sessionStorage y se redirige a /productos.

- El componente LoginView.vue usa usuariosService.getAll().
- Se comparan username y password contra la respuesta.
- Se guarda token en sessionStorage como `token\_\*` y username.
- La navegación usa router.beforeEach para proteger rutas.
- No existen roles ni permisos especiales en la aplicación actual.

5. Servicios API

El archivo src/services/api.js define dos servicios Axios para consumir MockAPI.

- usuariosService: getAll, getById, create, update, delete.
- productosService: getAll, create, update, delete.
- BASE_URL: la dirección base de MockAPI debe ajustarse al proyecto real.

6. Gestión de usuarios

La vista UsuariosView.vue permite crear, editar y eliminar usuarios usando modales Bootstrap.

- Crear usuario: abre modal y envía datos a `usuariosService.create`.
- Editar usuario: carga datos en el formulario y llama `usuariosService.update`.
- Eliminar usuario: confirma con modal y llama `usuariosService.delete`.
- Lista de usuarios en una tabla responsive con acciones de edición y borrado.
- Validaciones básicas: nombre, username, email y password.

7. Gestión de productos

La vista `ProductosView.vue` muestra productos en tarjetas y soporta imágenes por URL.

- Crear producto: formulario en modal con campos nombre, descripción, imagen, precio, stock y categoría.
- Editar producto: mismo formulario con valores cargados y validación.
- Eliminar producto: modal de confirmación antes de borrar.
- Imagen del producto: usa ``producto.imagen`` o un placeholder si la URL falla.
- Validación de imagen con función ``esUrlValida(url)``.
- Las tarjetas usan CSS global para mostrar la imagen completa sin recortes.

8. CSS y diseño

Los estilos globales se centralizan en `src/assets/global.css`. Allí se ajustan los modales, inputs, botones y la presentación de las tarjetas de producto.

- Estilos de ``glass-modal``, ``futuristic-input`` y ``futuristic-btn``.
- Ajustes para ``card-img-top`` y ``product-image`` que permiten mostrar la imagen completa.
- Responsive design para modal y tarjetas en pantallas pequeñas.

9. Flujo de uso típico

El profesor puede preguntar por el flujo completo desde el login hasta la edición de datos.

- 1) Usuario abre app y ve `/login`.
- 2) Ingresa credenciales y el sistema valida contra MockAPI.
- 3) Si es correcto, se redirige a `/productos`.
- 4) Desde la navbar puede ir a Usuarios o Productos.
- 5) En Usuarios: puede crear, editar y eliminar usuarios.
- 6) En Productos: puede crear, editar y eliminar productos, usando URL de imagen.
- 7) Al cerrar sesión se borra el token y se vuelve a `/login`.

10. Puntos importantes que puede preguntar el profesor

Estos son los aspectos más probables de examen:

- Cómo funciona el guard de rutas en `router/index.js`.
- Dónde se define la autenticación y cómo se guarda el token.
- Cómo se consumen los servicios de MockAPI desde `src/services/api.js`.
- Qué vista maneja el CRUD de usuarios y qué vista maneja el CRUD de productos.
- Cómo se valida la URL de imagen y cómo se muestra el placeholder cuando falla.
- Qué archivos contienen los estilos globales y cómo se evita CSS embebido.

11. Archivos clave y ubicación

Para una respuesta rápida en examen, estos son los archivos y su propósito principal:

- `src/main.js`: arranca Vue y monta el router.
- `src/App.vue`: navbar, estado de login y router-view.
- `src/router/index.js`: rutas y auth guard.
- `src/views/LoginView.vue`: formulario de login.
- `src/views/UsuariosView.vue`: tabla de usuarios y modales de CRUD.

- src/views/ProductosView.vue: tarjetas de productos, imágenes, modales de CRUD.
- src/services/api.js: definición de endpoints Axios.
- src/components/AlertaComponent.vue: alertas reutilizables.
- src/assets/global.css: estilos compartidos.

12. Dependencias principales

En package.json aparecen las librerías usadas en el proyecto.

- vue: framework principal.
- vue-router: enrutamiento SPA.
- axios: peticiones HTTP REST.
- @vitejs/plugin-vue y vite: para bundling y desarrollo.

13. Recomendaciones de revisión

Si el profesor pregunta por posibles mejoras o reglas de negocio, menciona esto:

- No hay roles, la aplicación solo protege rutas con autenticación simple.
- El login usa una comparación local de contraseña en el cliente; no es segura para producción.
- La API de MockAPI debe existir con rutas `/usuarios` y `/productos`.
- Las imágenes se cargan por URL y se controla el error con placeholder.